程设机考

单调性枚举

```cpp
template<typename M, typename I, typename R, typename U>
void increaseEnumerate(int s, int e,
                       const M& match,
                       const I& insert,
                       const R& remove,
                       const U& update) {
  for (int l = s, r = s; l <= e; ) {
    while (l == r || (r <= e && !match(l, r - 1))) insert(l, r++);
    if (match(l, r - 1)) update(l, r - 2);
    else update(l, r - 1);
    remove(l++, r);
  }
}
```

并查集

```cpp
template<bool Heuristic = true, bool PathCompression = true>
struct disjoint_set {
  vector<int> fa_or_sz;
  disjoint_set(int n) : fa_or_sz(n + 1, -1) {}
  int find(int u) {
    if constexpr (PathCompression) {
      if (fa_or_sz[u] < 0) return u;
      return fa_or_sz[u] = find(fa_or_sz[u]);
    } else {
      for ( ; fa_or_sz[u] >= 0; u = fa_or_sz[u]);
      return u;
    }
  }
  pair<int, int> unite (int u, int v) {
    int from = find(u), to = find(v);
    if (from == to) return {u, -1};
    if constexpr (Heuristic) {
      if (fa_or_sz[from] < fa_or_sz[to]) swap(from, to);
    }
    fa_or_sz[to] += fa_or_sz[from];
    fa_or_sz[from] = to;
    return {from, to};
  }
  bool is_united(int u, int v) { return find(u) == find(v); }
  int size(int u) { return -fa_or_sz[find(u)]; }
  int fa(int u) {return fa_or_sz[u] < 0 ? u : fa_or_sz[u]; }
  vector<vector<int>> group(int begin = 0, int end = -1) {
    if (end == -1) end = fa_or_sz.size() - 1;
    vector<vector<int>> res(fa_or_sz.size());
    for (int i = begin; i <= end; i ++) {
```

```
      res[find(i)].push_back(i);
    }
    res.erase(
      remove_if(res.begin(), res.end(), [](vector<int>& v){ return v.empty();}),
      res.end()
    );
    return res;
  }
};
```

wangzherongyao

```
#include <assert.h>

class Hero {
 public:
  double life;
  double baseAttack;
  Hero(double life, double baseAttack) :life(life), baseAttack(baseAttack) {};
  virtual bool attack(Hero& hero) = 0;

  double getCurrentLife() const {
    return life;
  }
  double getBaseDamage() const {
    return baseAttack;
  }
  void takeDamage(double value) {
    getHurt(value);
  }

 protected:
   virtual void getHurt(double value) = 0;
};

class Warrior : public Hero {
 public:
   Warrior(double life, double baseAttack) : Hero(life, baseAttack) {};
  bool attack(Hero& hero) override {
    hero.takeDamage(baseAttack);
    return hero.life <= 1e-8;
  }
protected:
  void getHurt(double value) override {
    value *= 0.8;
    if (life < value) life = 0;
    else life -= value;
  }
};

class Mage : public Hero{
public:
```

```
  Mage(double life, double baseAttack) : Hero(life, baseAttack) {};
  bool attack(Hero& hero) override {
    hero.takeDamage(baseAttack * 1.25);
    return hero.life <= 1e-8;
  }
protected:
  void getHurt(double value) override {
    if (life < value) life = 0;
    else life -= value;
  }
};

class Arthur : public Warrior {
public:
  Arthur(double life, double baseAttack) : Warrior(life,  baseAttack) {};
  bool attack(Hero& hero) override {
    if (Mage* m = dynamic_cast<Mage*>(&hero)) {
      m->takeDamage(baseAttack * 1.1);
    } else if (Warrior* w = dynamic_cast<Warrior*>(&hero)) {
      w->takeDamage(baseAttack * 0.9);
    }
    return hero.life <= 1e-8;
  }
};

class Angela : public Mage {
public:
  Angela(double life, double baseAttack) : Mage(life, baseAttack) {};
protected:
  void getHurt(double value) {
    if (life < 500.0) {
      if (life < value * 0.3) life = 0;
      else life -= value * 0.3;
    } else life -= value;
  }
};
```

duotai

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

class Spell {
    private:
        string scrollName;
    public:
        Spell(): scrollName("") { }
        Spell(string name): scrollName(name) { }
        virtual ~Spell() { }
        string revealScrollName() {
```

```
            return scrollName;
        }
};

class Fireball : public Spell {
    private: int power;
    public:
        Fireball(int power): power(power) { }
        void revealFirepower(){
            cout << "Fireball: " << power << endl;
        }
};

class Frostbite : public Spell {
    private: int power;
    public:
        Frostbite(int power): power(power) { }
        void revealFrostpower(){
            cout << "Frostbite: " << power << endl;
        }
};

class Thunderstorm : public Spell {
    private: int power;
    public:
        Thunderstorm(int power): power(power) { }
        void revealThunderpower(){
            cout << "Thunderstorm: " << power << endl;
        }
};

class Waterbolt : public Spell {
    private: int power;
    public:
        Waterbolt(int power): power(power) { }
        void revealWaterpower(){
            cout << "Waterbolt: " << power << endl;
        }
};

class SpellJournal {
    public:
        static string journal;
        static string read() {
            return journal;
        }
};
string SpellJournal::journal = "";

void counterspell(Spell *spell) {

  /* Enter your code here */
  if (Fireball* fireball = dynamic_cast<Fireball*>(spell)) {
```

```
    fireball->revealFirepower();

  } else if (Frostbite* frostbite = dynamic_cast<Frostbite*>(spell)) {

    frostbite->revealFrostpower();

  } else if (Thunderstorm* fireball = dynamic_cast<Thunderstorm*>(spell)) {

    fireball->revealThunderpower();

  } else if (Waterbolt* waterbolt = dynamic_cast<Waterbolt*>(spell)) {

    waterbolt->revealWaterpower();

  } else {
    string a = spell->revealScrollName(), b = SpellJournal::read();
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i ++) {
      for (int j = 1; j <= m; j ++) {
        if (a[i-1] == b[j-1]) dp[i][j] = dp[i - 1][j - 1] + 1;
        dp[i][j] = max(dp[i][j], max(dp[i - 1][j], dp[i][j - 1]));
      }
    }
    cout << dp[n][m] << endl;
  }
}

class Wizard {
    public:
        Spell *cast() {
            Spell *spell;
            string s; cin >> s;
            int power; cin >> power;
            if(s == "fire") {
                spell = new Fireball(power);
            }
            else if(s == "frost") {
                spell = new Frostbite(power);
            }
            else if(s == "water") {
                spell = new Waterbolt(power);
            }
            else if(s == "thunder") {
                spell = new Thunderstorm(power);
            }
            else {
                spell = new Spell(s);
                cin >> SpellJournal::journal;
            }
            return spell;
        }
```

```
};

int main() {
    int T;
    cin >> T;
    Wizard Arawn;
    while(T--) {
        Spell *spell = Arawn.cast();
        counterspell(spell);
    }
    return 0;
}
```